

Section 4.2: Sprint 2 Development Implementation

SEN5002 Agile Development and DevOps — ICBT Carpooling System Project Report

4.2.1 Sprint 2 Development Overview

Sprint 2 extends the foundational authentication and profile infrastructure established in Sprint 1 by introducing key ride-discovery and interaction capabilities—specifically, ride-offer search, public offer detail inspection, and join-request submission and status tracking. In alignment with updated domain requirements, the system operates without rigid role-based access restrictions (such as hardcoded Driver or Passenger account types). Instead, any authenticated ICBT user can publish ride offers as an offer owner and submit join requests for other users' offers as a requester. Security and access control are enforced dynamically through token-based authentication, entity ownership checks (userId and requesterUserId), and action-context validation. Sprint 2 seamlessly reuses the Sprint 1 Express router pattern, MongoDB native driver access architecture, JWT middleware, and unified UI styling system, ensuring architectural consistency and zero regression across the application lifecycle.

4.2.2 Implemented Development Items

Table 4.2.1 summarizes the core functional development items implemented during Sprint 2, mapping each item to its corresponding user story, technical implementation details, and verification outcomes.

Table 4.2.1: Sprint 2 Implemented Development Items

Implemented Item	Related User Story	Implementation Detail	Expected & Actual Outcome
Searchable Ride-Offer Listing	US-13: Search active ride offers	Built GET /api/ride-offers/search endpoint utilizing MongoDB \$and / \$or query construction with \$regex case-insensitive pattern matching for origin, destination, date, and time window. Excludes offers where availableSeats <= 0 or status !== Active. Integrated custom PickerField UI components.	Expected: Eligible active offers matching search criteria returned. Actual: Verified via API tests (TDD-S2-01, TDD-S2-02) and UI search component integration.
Public Offer Detail Viewing	US-14: View public offer details	Developed GET /api/ride-offers/public/:id endpoint returning complete route, schedule, seat count, and status information. Applies privacy projection (toPublicRideOffer serializer) restricting owner metadata strictly to firstName and lastName.	Expected: Offer details returned with sensitive owner contact info omitted. Actual: Verified via unit test TDD-S2-03 and public offer detail modal.
Join-Request Creation	US-15: Create join request	Created POST /api/join-requests endpoint accepting rideOfferId and optional requestNote. Validates offer existence, status (Active), seat availability (> 0), self-request restriction, and duplicate request status. Inserts record with status: Pending.	Expected: Valid request creates a Pending record containing rideOfferId, requesterUserId, ownerUserId. Actual: Verified via API test TDD-S2-04.
Business Rule Enforcement & Validation Guards	US-15: Join request business rules	Implemented validation guard functions in validators.js evaluating request context against state rules: (1) block duplicate active requests, (2) block self-requests (requesterUserId === ownerUserId), and (3) reject requests for inactive, full (0 seats), or cancelled offers.	Expected: Invalid requests rejected with HTTP status codes (409 Conflict, 422 Unprocessable Entity). Actual: Verified via unit tests TDD-S2-05, TDD-S2-06, TDD-S2-07a/b/c.
Requester Request-Status Viewing	US-16: View own request status	Implemented GET /api/join-requests/mine endpoint fetching all join requests where requesterUserId matches the authenticated token payload. Embeds offer route summary and owner name for display in the requester dashboard.	Expected: Users view only their own request history and current statuses (Pending, Accepted, Rejected). Actual: Verified via unit test TDD-S2-08.
Regression Stability & UI Consistency	US-13 to US-16: System stability	Maintained existing Sprint 1 brand tokens, typography, and button styling in styles.css. Reused AppShell and custom pickers. Re-executed full test suite.	Expected: All Sprint 1 auth/profile/ride functionality remains operational. Actual: 47/47 existing tests and 24 new Sprint 2 tests pass (71/71 total).

4.2.3 API, Database, and Security Implementation

Table 4.2.2 details the technical implementation of Sprint 2 API routes, MongoDB query structures, validation rules, security checks, and UI integration patterns.

Table 4.2.2: Sprint 2 Technical Architecture & Security Matrix

Architecture Domain	Implementation Specification
API Routes & Controllers	• GET /api/ride-offers/search — Params: origin, destination, date, timeWindow. Returns array of eligible active offers. • GET /api/ride-offers/public/:id — Param: id. Returns sanitized public offer summary. • POST /api/join-requests — Body: { rideOfferId, requestNote }. Creates pending join request. • GET /api/join-requests/mine — Query: authenticated user requests. Returns list of requester's join requests.
MongoDB Collections & Queries	• rideOffers collection: Query filters status: 'Active', availableSeats: { \$gt: 0 }, and \$regex queries on origin and destination. • joinRequests collection: Schema stores _id, rideOfferId, requesterUserId, ownerUserId, status ('Pending', 'Accepted', 'Rejected'), requestNote, requestedAt, updatedAt. Indexed on { requesterUserId: 1 } and { rideOfferId: 1, requesterUserId: 1 }.
JoinRequest Validation	Implemented validateJoinRequest(payload, offer, existingRequests, requesterUserId) in validators.js: 1. Checks if target offer exists (404). 2. Verifies offer.status === 'Active' and offer.availableSeats > 0 (422). 3. Verifies requesterUserId !== offer.userId to block self-requests (422). 4. Checks joinRequests collection for existing request with status: 'Pending' by the same requester (409).
Authentication & Authorization	• All Sprint 2 endpoints protected by requireAuth middleware verifying JWT Bearer token. • Ownership & action-context authorization: GET /api/join-requests/mine restricts queries to requesterUserId === req.user._id. POST /api/join-requests extracts requesterUserId directly from verified token context.
Privacy Controls	• toPublicRideOffer(offer, ownerProfile) serializer strips userId, phone number, email, studentStaffId, and vehicle details before transmitting offer data across public endpoints. • Public views display only owner.firstName and owner.lastName.
Error Handling	Standardized JSON error responses with explicit HTTP status codes: 400 (Validation Error), 401 (Unauthorized), 404 (Not Found), 409 (Conflict / Duplicate), 422 (Unprocessable Entity / Business Rule Failure), 500 (Internal Error).
UI Integration	• SearchRidePage.jsx: Search bar with PickerField components for Date and Time Window matching RideCreatePage.jsx styling. • OfferDetailPage.jsx: Modal view presenting public offer details and 'Send Join Request' interface. • MyRequestsPage.jsx: Requester dashboard displaying request status cards with visual status badges.

4.2.4 Developer Unit / API Testing

To ensure software quality prior to QA handoff, automated developer tests were written and executed using Jest and Supertest. Tests were structured following Test-Driven Development (TDD) principles and unit test coverage criteria. Table 4.2.3 outlines the developer test matrix.

Table 4.2.3: Developer Automated Test Execution Matrix

Test ID	Feature / Module	Test Focus	Expected Result	Evidence Placeholder
TDD-S2-01	Ride Search	Search with valid route & time-window criteria	Eligible active ride offers returned with HTTP 200	[Figure S2-DEV-02]
TDD-S2-02	Ride Search	Search with non-matching criteria	Empty array returned with descriptive message	[Figure S2-DEV-02]
TDD-S2-03	Offer Details	Public detail endpoint retrieval	Returns route, time, seats, status, and limited owner name	[Figure S2-DEV-02]
TDD-S2-04	Join Request	Valid join request creation	Creates Pending JoinRequest in DB with HTTP 201	[Figure S2-DEV-02]
TDD-S2-05	Join Request	Duplicate pending request attempt	Request blocked with HTTP 409 Conflict	[Figure S2-DEV-02]
TDD-S2-06	Join Request	Self-request attempt by offer owner	Request blocked with HTTP 422 Unprocessable Entity	[Figure S2-DEV-02]
TDD-S2-07	Join Request	Request on inactive/full/cancelled offer	Request rejected with HTTP 422 Unprocessable Entity	[Figure S2-DEV-02]
TDD-S2-08	Request Status	View authenticated user's own requests	Returns list filtered strictly by requesterUserId	[Figure S2-DEV-02]
TDD-S2-09	Authorization	Unauthenticated request to protected API	Access denied with HTTP 401 Unauthorized	[Figure S2-DEV-02]

Test ID	Feature / Module	Test Focus	Expected Result	Evidence Placeholder
UT-S2-01	Search Builder	Input normalization (case & whitespace)	Regex matches origin/destination regardless of casing	[Figure S2-DEV-02]
UT-S2-02	Active Filter	Offer eligibility filter	Inactive/cancelled offers excluded from search results	[Figure S2-DEV-02]
UT-S2-03	Seat Availability	Zero-seat offer filter	Offers with availableSeats = 0 excluded	[Figure S2-DEV-02]
UT-S2-04	Offer Detail	Response schema verification	All display fields (origin, destination, date) present	[Figure S2-DEV-02]
UT-S2-05	Privacy Control	Privacy projection check	phoneNumber, email, studentStaffId omitted	[Figure S2-DEV-02]
UT-S2-06	Validation	DB insert persistence	Verified joinRequests collection insertion	[Figure S2-DEV-02]
UT-S2-07	Validation	Duplicate state check	Second Pending request correctly prevented	[Figure S2-DEV-02]
UT-S2-08	Validation	Self-request logic check	requesterUserId === ownerUserId check enforced	[Figure S2-DEV-02]
UT-S2-09	Validation	Offer state check	Non-active offer states rejected cleanly	[Figure S2-DEV-02]
UT-S2-10	Request Status	User isolation check	Requester A cannot see Requester B's requests	[Figure S2-DEV-02]
UT-S2-11	Security Guard	Missing token handling	Requests without Bearer header rejected with 401	[Figure S2-DEV-02]
UT-S2-12	Regression	Sprint 1 core regression suite	All 24 Sprint 1 auth/profile/ride tests pass cleanly	[Figure S2-DEV-02]

4.2.5 Development Evidence References

The following figures serve as evidence placeholders for code commits, automated test execution, continuous integration, and runtime user interface verification:

- Figure S2-DEV-01: GitHub Repository Commit & Pull Request Evidence — Showing Sprint 2 feature branch merge history, commit messages, and version control trace for ride search and join request features.
- Figure S2-DEV-02: Local Automated Unit & API Test Execution Result — Terminal output confirming 71 passed tests across 5 test suites (sprint1-api.test.js, sprint2-api.test.js, sprint2-tdd-spec.test.js, auth.test.js, validators.test.js).
- Figure S2-DEV-03: Continuous Integration (CI) Workflow Result — GitHub Actions pipeline run confirming successful build, linting, and automated Jest test execution on push.
- Figure S2-DEV-04: Sprint 2 Runtime UI Screens — Application screenshots demonstrating: (a) Find a Ride Page with custom pickers, (b) Public Offer Detail View, (c) Join Request Modal & Submission, and (d) My Requests Page.
- Figure S2-DEV-05: Browser Developer Tools Network Evidence — Network tab capture confirming correct HTTP request payloads and responses (GET /api/ride-offers/search, POST /api/join-requests, GET /api/join-requests/mine).
- Figure S2-DEV-06: Security & Ownership Rule Verification Log — Server response logs verifying HTTP 409 Conflict on duplicate requests, 422 Unprocessable Entity on self-requests, and 401 Unauthorized on missing tokens.

4.2.6 Development Summary

Sprint 2 successfully delivered the complete ride-discovery and join-request workflow foundation for the ICBT Carpooling System. By enforcing role-free, context-based access control, the system enables any authenticated user to seamlessly discover active ride offers, inspect privacy-safe offer details, and submit join requests. All core business rules—including duplicate request prevention, self-request blocking, seat availability checks, and requester data isolation—were fully implemented and validated through developer unit and API tests. With 71/71 automated tests passing cleanly and zero regression across Sprint 1 functionality, Sprint 2 provides a robust, production-ready foundation for Sprint 3, where offer owners will evaluate and process incoming join requests.